

LAPORAN TUGAS BESAR

ALGORITMA DAN PEMROGRAMAN 2

Aplikasi Pengelola Kata Sandi Pribadi (SecurePass)



Disusun oleh:

Iqbal Rizqullah Jaffan - 108072500077

Kelvin Aditya Gazannova - 108072500011

Kelas:

IF-05-03

Dosen Pengampu:

Dr. Tanzilal Mustaqim, S.Kom., M.Kom.

PROGRAM STUDI INFORMATIKA

FAKULTAS INFORMATIKA

UNIVERSITAS TELKOM

TAHUN AKADEMIK 2025/2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas berkat, rahmat, dan karunia-Nya sehingga penulis dapat menyelesaikan laporan Tugas Besar mata kuliah Algoritma dan Pemrograman 2 ini dengan baik dan tepat waktu.

Laporan ini disusun sebagai bentuk pertanggungjawaban atas pengerjaan tugas besar yang merupakan salah satu syarat kelulusan mata kuliah Algoritma dan Pemrograman 2. Melalui tugas besar ini, penulis berkesempatan menerapkan konsep algoritma, pemrograman modular, dan perancangan aplikasi dalam menyelesaikan studi kasus yang diberikan.

Pada kesempatan ini penulis mengucapkan terima kasih kepada:

- Bapak/Ibu Dosen Pengampu mata kuliah Algoritma dan Pemrograman 2 atas bimbingan dan arahan yang diberikan.
- Rekan-rekan satu kelompok yang telah bekerja sama dalam penyusunan laporan dan pengembangan program.
- Pihak-pihak lain yang turut mendukung penyelesaian tugas besar ini.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat penulis harapkan demi perbaikan di masa mendatang. Semoga laporan ini dapat memberikan manfaat bagi pembaca.

Surabaya, 25 Juni 2026

Kelvin Aditya Gazannova/Iqbal Rizqulloh Jaffan

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB I	
PENDAHULUAN.....	4
1.1 Ringkasan Tugas Besar.....	4
1.2 Tujuan.....	4
BAB II	
DESKRIPSI TUGAS BESAR.....	6
2.1 Gambaran Umum Aplikasi.....	6
2.2 Fitur-Fitur Aplikasi.....	6
2.3 Flowchart Fitur.....	7
BAB III	
PENJELASAN PROGRAM.....	12
3.1 Struktur Program Modular.....	12
1. Fungsi Utama (Main Function).....	12
2. Modul Manajemen Data Akun (Fitur CRUD).....	12
3. Modul Pencarian dan Pengurutan (Searching & Sorting).....	12
4. Modul Validasi dan Statistik.....	13
3.2 Pseudocode.....	14
3.3 Penjelasan Kode Program.....	20
1. Pengolahan Struct dan Array/Slice.....	20
2. Perulangan (Looping).....	21
3. Logika Percabangan (Conditional).....	23
BAB IV	
TANTANGAN DAN SOLUSI.....	26
3.4 Kendala yang Dihadapi dan Solusi Penanganannya.....	26
1. Perancangan Validasi pada Algoritma Pencarian (Binary Search).....	26
2. Runtime Error Saat Implementasi Penghapusan Data (Slicing).....	26
3. Masalah Validasi Input pada Tipe Data String yang Mengandung Spasi.....	27
4. Kompleksitas Logika Klasifikasi Kekuatan Kata Sandi.....	27
BAB V	
KESIMPULAN DAN REKOMENDASI.....	28
5.1 Kesimpulan.....	28
5.2 Rekomendasi.....	28
TAUTAN GITHUB.....	30

REFERENSI.....	31
-----------------------	-----------

BAB I

PENDAHULUAN

1.1 Ringkasan Tugas Besar

Aplikasi SecurePass ini dikembangkan sebagai pemenuhan Tugas Besar untuk mata kuliah Algoritma Pemrograman 2 di Fakultas Informatika, School of Computing, Telkom University. Proyek ini berfokus pada penerapan struktur data (*struct*), operasi CRUD (*Create, Read, Update, Delete*), serta implementasi algoritma pencarian (*Sequential* dan *Binary Search*) dan pengurutan (*Selection* dan *Insertion Sort*) di dalam bahasa pemrograman Go.

Permasalahan yang Ingin Diselesaikan

Aplikasi ini dirancang untuk mengatasi beberapa permasalahan utama dalam manajemen keamanan digital personal, antara lain:

- **Kesulitan Mengelola Informasi Login:** Banyaknya akun layanan digital yang dimiliki individu membuat mereka kesulitan untuk mengingat semua kombinasi nama pengguna dan kata sandi.
- **Risiko Keamanan Akses Digital:** Pengguna membutuhkan media penyimpanan data login yang aman dan bersifat lokal guna meminimalisir risiko kebocoran data di jaringan.
- **Kurangnya Kesadaran terhadap Kekuatan Sandi:** Pengguna sering kali tidak menyadari jika kata sandi yang mereka gunakan terlalu lemah. Oleh karena itu, diperlukan sistem yang dapat mendeteksi, mengklasifikasikan, serta menampilkan statistik kekuatan kata sandi secara otomatis.
- **Efisiensi Pencarian dan Pengorganisasian Data:** Seiring bertambahnya jumlah akun, pencarian data secara manual menjadi tidak efisien. Diperlukan mekanisme pengurutan data secara alfabetis serta pencarian yang cepat dan terstruktur berdasarkan nama layanan tertentu.

1.2 Tujuan

Tuliskan tujuan pengembangan tugas besar, antara lain:

- **Menerapkan Konsep Algoritma dan Pemrograman**
Mengimplementasikan teori dan konsep yang telah dipelajari dalam mata kuliah Algoritma Pemrograman 2, khususnya mengenai struktur data (*struct*), operasi CRUD, serta algoritma pengurutan (*sorting*) dan pencarian (*searching*) menggunakan bahasa pemrograman Go.
- **Mengembangkan Aplikasi yang Sesuai dengan Studi Kasus**
Membangun aplikasi pengelola kata sandi bernama **SecurePass** yang mampu menjawab kebutuhan spesifikasi teknis untuk mengamankan data akun layanan, nama pengguna, dan kata sandi secara lokal dan terstruktur.
- **Melatih Kemampuan Analisis, Perancangan, dan Implementasi Program**

Mengasah kompetensi mahasiswa dalam menganalisis permasalahan manajemen keamanan digital , merancang solusi berbasis algoritma yang efisien (seperti *Binary Search* dan *Insertion/Selection Sort*), serta mengimplementasikannya ke dalam bentuk kode program yang fungsional dan interaktif.

BAB II

DESKRIPSI TUGAS BESAR

2.1 Gambaran Umum Aplikasi

Aplikasi yang dikembangkan dalam tugas besar ini adalah Aplikasi Pengelola Kata Sandi Pribadi (SecurePass), sebuah program berbasis *Command Line Interface* (CLI) yang dibangun menggunakan bahasa pemrograman Go (Golang). Fungsi utama dari aplikasi ini adalah untuk menyimpan, mengelola (menambah, mengubah, menghapus), mencari, serta mengurutkan data informasi login akun—seperti nama layanan, alamat email, kata sandi, dan tanggal pembaruan—secara lokal dan aman, sekaligus menyediakan fitur klasifikasi dan statistik kekuatan kata sandi. Target pengguna dari aplikasi ini adalah individu atau pengguna personal yang ingin mengamankan dan mengorganisasikan akses digital mereka terhadap berbagai akun layanan dengan lebih efisien.

2.2 Fitur-Fitur Aplikasi

- **Fitur 1 (Tambah Akun)**

Memungkinkan pengguna untuk memasukkan data akun layanan baru ke dalam sistem, yang meliputi nama layanan, alamat email, kata sandi, dan tanggal pembaruan terakhir.

- **Fitur 2 (Edit Akun)**

Memungkinkan pengguna untuk mengubah data alamat email, kata sandi, dan tanggal pembaruan pada akun layanan yang sudah tersimpan sebelumnya berdasarkan nama layanan yang dicari.

- **Fitur 3 (Hapus Akun)**

Memungkinkan pengguna untuk menghapus data akun layanan tertentu dari sistem berdasarkan nama layanan yang diinputkan.

- **Fitur 4 (Cari Akun)**

Memungkinkan pengguna untuk menemukan informasi akun secara cepat berdasarkan nama layanan dengan memilih salah satu dari dua metode pencarian, yaitu *Sequential Search* atau *Binary Search*.

- **Fitur 5 (Urutkan Akun)**

Memungkinkan pengguna untuk mengurutkan seluruh data akun yang tersimpan berdasarkan nama layanan secara alfabetis menggunakan algoritma *Selection Sort* atau *Insertion Sort*.

- **Fitur 6 (Tampilkan Data & Statistik)**

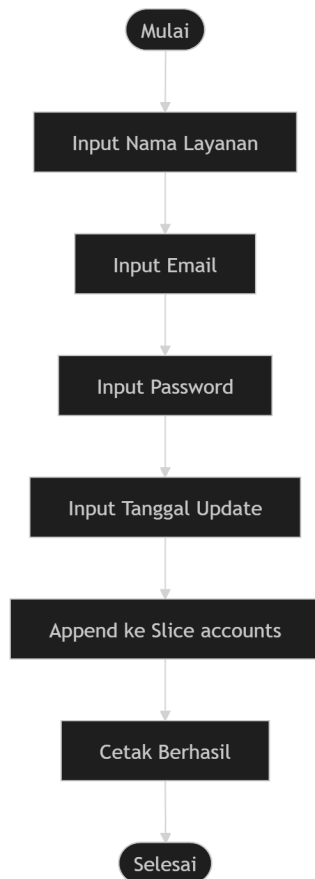
Memungkinkan pengguna untuk melihat seluruh daftar akun yang tersimpan beserta statistik jumlah total akun dan klasifikasi kekuatan kata sandinya (Lemah, Sedang, atau Kuat) yang dihitung secara otomatis oleh sistem.

- **Fitur 7 (Keluar)**

Memungkinkan pengguna untuk menghentikan sesi program dan keluar dari aplikasi SecurePass secara aman.

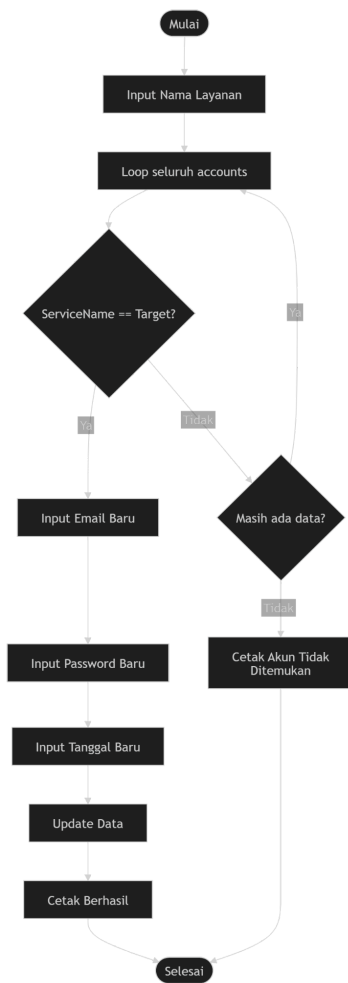
2.3 Flowchart Fitur

- Flowchart Menambah Akun



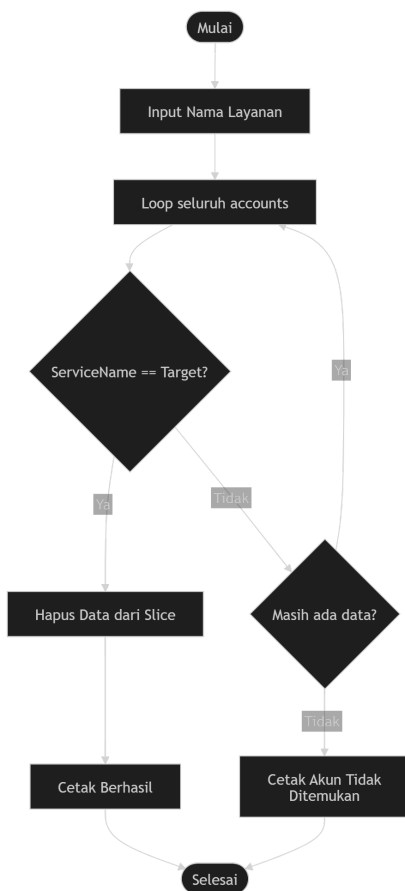
Program meminta pengguna memasukkan informasi berupa nama layanan, email, password, dan tanggal pembaruan terakhir. Setelah seluruh data berhasil dimasukkan, data tersebut disimpan dalam sebuah variabel bertipe Account kemudian ditambahkan ke dalam slice accounts menggunakan fungsi `append()`. Setelah proses penambahan selesai, program menampilkan pesan bahwa akun berhasil ditambahkan dan mengembalikan kendali ke menu utama.

- Flowchart Edit Akun



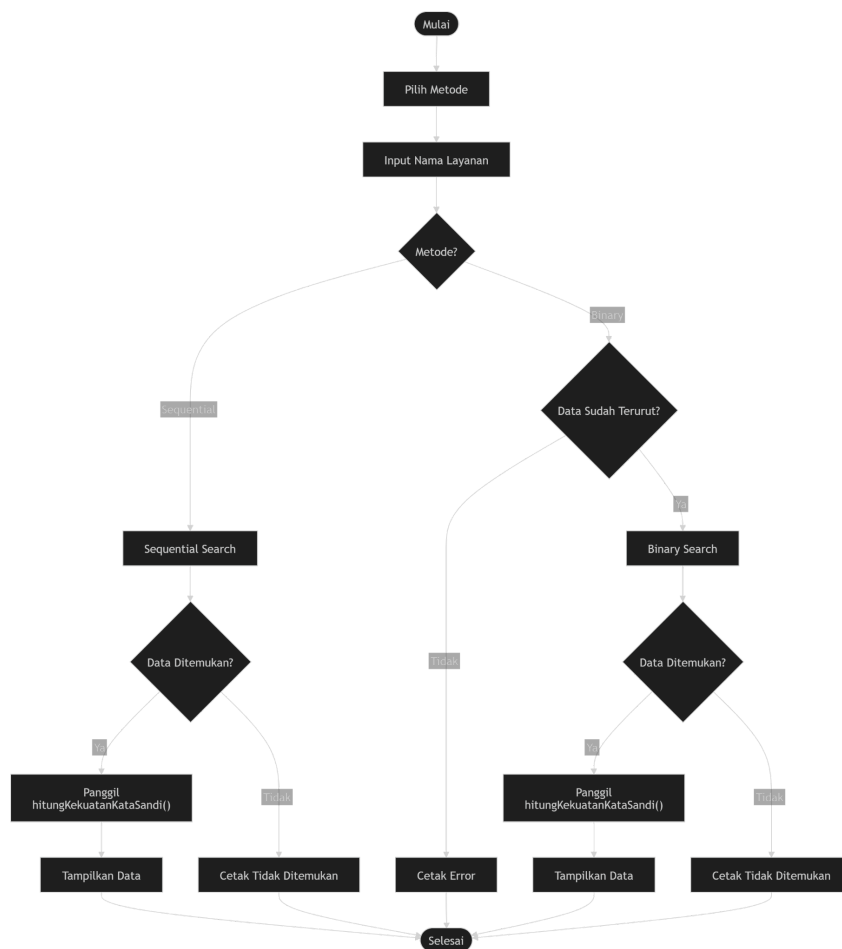
Program terlebih dahulu meminta pengguna memasukkan nama layanan yang ingin diubah. Selanjutnya program melakukan pencarian secara berurutan (sequential search) dengan memeriksa setiap elemen dalam slice accounts. Apabila nama layanan yang dimasukkan ditemukan, pengguna diminta memasukkan email baru, password baru, dan tanggal pembaruan baru untuk menggantikan data sebelumnya. Setelah data diperbarui, program menampilkan pesan bahwa proses edit berhasil. Sebaliknya, jika seluruh data telah diperiksa namun nama layanan tidak ditemukan, program akan menampilkan pesan bahwa akun tidak ditemukan.

- Flowchart Hapus Akun



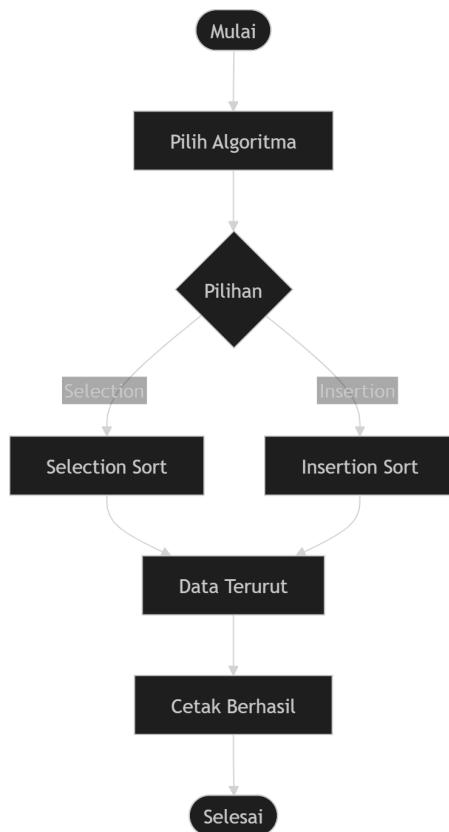
Program meminta pengguna memasukkan nama layanan yang akan dihapus, kemudian melakukan pencarian satu per satu terhadap seluruh data akun. Apabila data yang dicari ditemukan, program menghapus akun tersebut dari slice accounts menggunakan operasi pemotongan slice (`append(accounts[:i], accounts[i+1:]...)`). Setelah penghapusan berhasil dilakukan, program menampilkan pesan konfirmasi kepada pengguna. Namun, jika hingga akhir proses pencarian akun tidak ditemukan, program akan memberikan informasi bahwa data akun yang dimaksud tidak tersedia.

- Flowchart Cari Akun



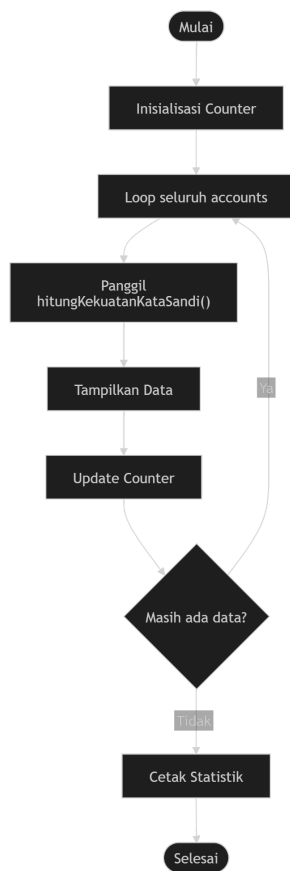
Flowchart ini menggambarkan proses pencarian akun yang menyediakan dua metode pencarian, yaitu Sequential Search dan Binary Search. Setelah pengguna memilih metode pencarian dan memasukkan nama layanan yang dicari, program akan menjalankan algoritma sesuai pilihan pengguna. Pada Sequential Search, program memeriksa setiap data akun secara berurutan hingga menemukan data yang sesuai. Sedangkan pada Binary Search, program terlebih dahulu memastikan bahwa data telah diurutkan berdasarkan nama layanan. Jika data belum terurut, program akan menampilkan pesan kesalahan. Apabila data sudah terurut, proses pencarian dilakukan dengan membagi ruang pencarian menggunakan variabel low, high, dan mid hingga data ditemukan atau dipastikan tidak ada. Jika akun berhasil ditemukan, program akan memanggil fungsi `hitungKekuatanKataSandi()` untuk menentukan tingkat keamanan password sebelum menampilkan informasi akun kepada pengguna.

- Flowchart Urutkan Akun



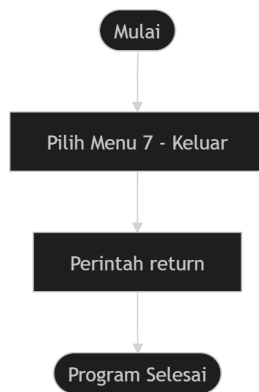
Pengguna diberikan pilihan untuk menggunakan algoritma Selection Sort atau Insertion Sort. Jika memilih Selection Sort, program akan mencari elemen dengan nilai terkecil pada bagian data yang belum terurut, kemudian menukarnya ke posisi yang benar hingga seluruh data tersusun. Apabila memilih Insertion Sort, program akan mengambil setiap elemen dan menyisipkannya ke posisi yang tepat pada bagian data yang telah terurut. Setelah proses pengurutan selesai, program menampilkan pesan bahwa data berhasil diurutkan.

- Flowchart Tampilkan data dan statistik



Program terlebih dahulu menginisialisasi penghitung untuk kategori password Lemah, Sedang, dan Kuat. Selanjutnya program melakukan perulangan terhadap seluruh data akun yang tersimpan. Untuk setiap akun, program memanggil fungsi `hitungKekuatanKataSandi()` untuk menentukan tingkat keamanan password, kemudian menampilkan informasi akun beserta hasil klasifikasinya. Pada saat yang sama, program juga menambahkan nilai penghitung sesuai kategori password yang diperoleh. Setelah seluruh data selesai diproses, program menampilkan jumlah total akun serta jumlah password yang termasuk kategori lemah, sedang, dan kuat sehingga pengguna dapat mengetahui tingkat keamanan keseluruhan data yang tersimpan.

- Flowchart Keluar



Ketika pengguna memilih menu 7 (Keluar) pada menu utama, program akan menjalankan perintah return yang terdapat pada fungsi main(). Perintah ini menghentikan perulangan menu utama sehingga program tidak lagi menampilkan menu atau menerima input dari pengguna. Setelah perulangan berakhir, aplikasi akan langsung selesai dijalankan dan seluruh proses dihentikan..

BAB III

PENJELASAN PROGRAM

3.1 Struktur Program Modular

1. Fungsi Utama (Main Function)

- **func main()**
 - **Peran:** Berperan sebagai pintu masuk utama (*entry point*) program. Fungsi ini mengatur alur jalannya aplikasi secara keseluruhan dengan menampilkan menu interaktif secara berulang (*looping*) dan mengarahkan pengguna ke fitur yang dipilih menggunakan percabangan switch-case.

2. Modul Manajemen Data Akun (Fitur CRUD)

- **func addAccount()**
 - **Peran:** Prosedur untuk menambahkan data akun baru. Fungsi ini menerima input nama layanan, email, kata sandi, dan tanggal dari pengguna, lalu memasukkannya ke dalam *array/slice* global accounts.
- **func editAccount()**
 - **Peran:** Prosedur untuk memperbarui data akun yang sudah ada. Fungsi ini mencari akun berdasarkan nama layanan yang diinputkan pengguna. Jika ditemukan, pengguna dapat memperbarui email, kata sandi, dan tanggal pembaruan terbarunya.
- **func deleteAccount()**
 - **Peran:** Prosedur untuk menghapus akun dari sistem. Fungsi ini mencari nama layanan tertentu, lalu menghapus elemen akun tersebut dari *slice* accounts menggunakan teknik *slicing*.

3. Modul Pencarian dan Pengurutan (Searching & Sorting)

- **func searchMenu()**
 - **Peran:** Prosedur untuk menangani fitur pencarian akun berdasarkan nama layanan. Di dalamnya terdapat implementasi dua algoritma: **Sequential Search** (pencarian berurutan langsung) dan **Binary Search** (pencarian bagi-dua yang membutuhkan validasi apakah data sudah terurut atau belum).
- **func sortMenu()**
 - **Peran:** Prosedur untuk mengurutkan daftar akun secara alfabetis berdasarkan nama layanan. Pengguna diberi opsi untuk memilih algoritma pengurutan yang ingin digunakan, yaitu antara **Selection Sort** atau **Insertion Sort**.

4. Modul Validasi dan Statistik

- **func hitungKekuatanKataSandi(kataSandi string) string**
 - **Peran:** Fungsi yang mengembalikan nilai (*return function*) berupa status kekuatan kata sandi ("Lemah", "Sedang", atau "Kuat"). Fungsi ini menghitung skor berdasarkan kriteria panjang karakter, keberadaan huruf besar, huruf kecil, angka, dan simbol khusus.
- **func displayAccounts()**
 - **Peran:** Prosedur untuk menampilkan seluruh data akun yang tersimpan di dalam sistem ke layar. Selain menampilkan data mentah, fungsi ini juga memanggil `hitungKekuatanKataSandi` untuk menghitung dan menampilkan statistik total akun serta klasifikasi jumlah kata sandi berdasarkan tingkat kekuatannya.

3.2 Pseudocode

ALGORITHM tambahAkun

procedure addAccount()

import: accounts (sebagai variabel global / slice dinamis)

kamus:

acc : Account

algoritma:

output("Nama Layanan: ")

input(acc.ServiceName)

output("Email: ")

input(acc.Email)

output("Password: ")

input(acc.Password)

output("Tanggal (DD-MM-YYYY): ")

input(acc.LastUpdate)

accounts <- append(accounts, acc)

output("Akun berhasil ditambahkan!")

endprocedure

ALGORITHM editAkun

procedure editAccount()

import: accounts (sebagai variabel global / slice dinamis)

kamus:

name : string

i : integer

algoritma:

output("Masukkan nama layanan yang ingin diedit: ")

input(name)

for i <- 0 to length(accounts) - 1 do

if accounts[i].ServiceName == name then

output("Email baru: ")

input(accounts[i].Email)

output("Password baru: ")

input(accounts[i].Password)

output("Tanggal baru: ")

input(accounts[i].LastUpdate)

output("Data berhasil diubah!")

exit procedure

endif

endfor

output("Akun tidak ditemukan.")

endprocedure

ALGORITHM hapusAkun

procedure deleteAccount()

import: accounts (sebagai variabel global / slice dinamis)

kamus:

name : string

i : integer

algoritma:

output("Masukkan nama layanan yang ingin dihapus: ")

input(name)

for i <- 0 to length(accounts) - 1 do

```

    if accounts[i].ServiceName == name then
        accounts <- append(accounts[:i], accounts[i+1:]...)
        output("Akun berhasil dihapus!")
        exit procedure
    endif
endfor
output("Akun tidak ditemukan.")
endprocedure

```

ALGORITHM SelectionSort(accounts)

kamus:

n, i, j, minIdx : integer

algoritma:

```

n <- length(accounts)
for i <- 0 to n - 2 do
    minIdx <- i
    for j <- i + 1 to n - 1 do
        if accounts[j].ServiceName < accounts[minIdx].ServiceName then
            minIdx <- j
        endif
    endfor
    // Tukar posisi elemen
    swap(accounts[i], accounts[minIdx])
endfor
END

```

ALGORITHM InsertionSort(accounts)

kamus:

n, i, j : integer

key : Account

algoritma:

```

n <- length(accounts)
for i <- 1 to n - 1 do

```

```

    key <- accounts[i]
    j <- i - 1
    while j >= 0 and accounts[j].ServiceName > key.ServiceName do
        accounts[j + 1] <- accounts[j]
        j <- j - 1
    endwhile
    accounts[j + 1] <- key
endfor
END

```

ALGORITHM SequentialSearch(accounts, target)

kamus:

n, i : integer

algoritma:

n <- length(accounts)

for i <- 0 to n - 1 do

if accounts[i].ServiceName == target then

return i

endif

endfor

return -1

END

ALGORITHM BinarySearch(accounts, target)

kamus:

low, high, mid : integer

algoritma:

low <- 0

high <- length(accounts) - 1

while low <= high do

mid <- (low + high) / 2

if accounts[mid].ServiceName == target then

return mid

```

    else if accounts[mid].ServiceName < target then
        low <- mid + 1
    else
        high <- mid - 1
    endif
endwhile
return -1
END

```

ALGORITHM hitungKekuatanKataSandi

```
function hitungKekuatanKataSandi(kataSandi : string) -> string
```

kamus:

```
    panjang, hurufBesar, hurufKecil, angka, simbol, skor, i : integer
```

```
    c : char
```

algoritma:

```
    panjang <- length(kataSandi)
```

```
    hurufBesar <- 0; hurufKecil <- 0; angka <- 0; simbol <- 0
```

```
    for i <- 0 to panjang - 1 do
```

```
        c <- kataSandi[i]
```

```
        if c >= 'A' and c <= 'Z' then
```

```
            hurufBesar <- hurufBesar + 1
```

```
        else if c >= 'a' and c <= 'z' then
```

```
            hurufKecil <- hurufKecil + 1
```

```
        else if c >= '0' and c <= '9' then
```

```
            angka <- angka + 1
```

```
        else
```

```
            simbol <- simbol + 1
```

```
        endif
```

```
    endfor
```

```
    skor <- 0
```

```
    if panjang >= 8 then skor <- skor + 1 endif
```

```

if panjang >= 12 then skor <- skor + 1 endif
if hurufBesar > 0 then skor <- skor + 1 endif
if hurufKecil > 0 then skor <- skor + 1 endif
if angka > 0 then skor <- skor + 1 endif
if simbol > 0 then skor <- skor + 1 endif

if skor <= 2 then
    return "Lemah"
else if skor <= 4 then
    return "Sedang"
else
    return "Kuat"
endif
endfunction

```

ALGORITHM tampilkanDataDanStatistik

procedure displayAccounts()

import: accounts (sebagai variabel global)

kamus:

lemahCount, sedangCount, kuatCount, i : integer

statusKekuatan : string

algoritma:

lemahCount <- 0; sedangCount <- 0; kuatCount <- 0

output("--- Daftar Akun & Kekuatan Kata Sandi ---")

for i <- 0 to length(accounts) - 1 do

statusKekuatan <- hitungKekuatanKataSandi(accounts[i].Password)

output("Layanan: ", accounts[i].ServiceName, " | Email: ", accounts[i].Email, " | Kekuatan: ", statusKekuatan)

if statusKekuatan == "Lemah" then

lemahCount <- lemahCount + 1

else if statusKekuatan == "Sedang" then

```

        sedangCount <- sedangCount + 1
    else if statusKekuatan == "Kuat" then
        kuatCount <- kuatCount + 1
    endif
endfor

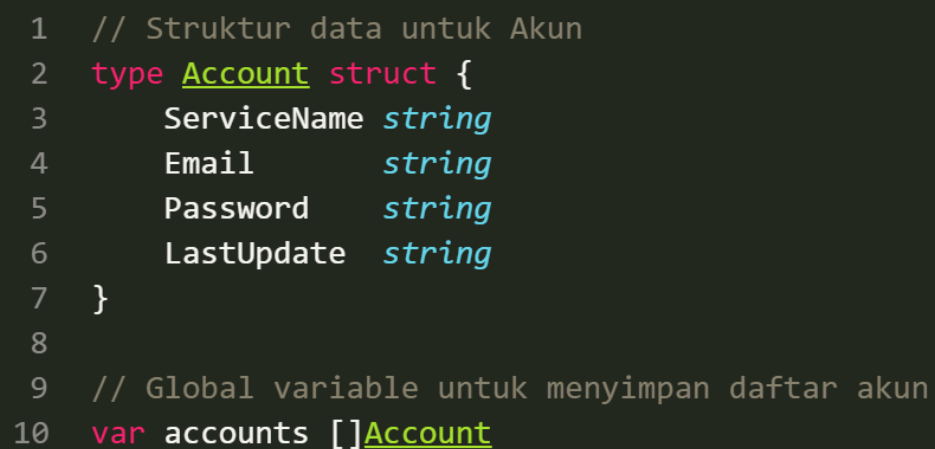
output("=== Statistik Total Akun: ", length(accounts), " ===")
output("Jumlah Sandi Lemah : ", lemahCount)
output("Jumlah Sandi Sedang : ", sedangCount)
output("Jumlah Sandi Kuat : ", kuatCount)
endprocedure

```

3.3 Penjelasan Kode Program

1. Pengolahan Struct dan Array/Slice

Bagian ini mendefinisikan struktur data utama yang digunakan untuk menampung informasi akun serta variabel global bertipe *slice* untuk menyimpan kumpulan data secara dinamis.



```

1  // Struktur data untuk Akun
2  type Account struct {
3      ServiceName string
4      Email       string
5      Password    string
6      LastUpdate  string
7  }
8
9  // Global variable untuk menyimpan daftar akun
10 var accounts []Account

```

Penjelasan:

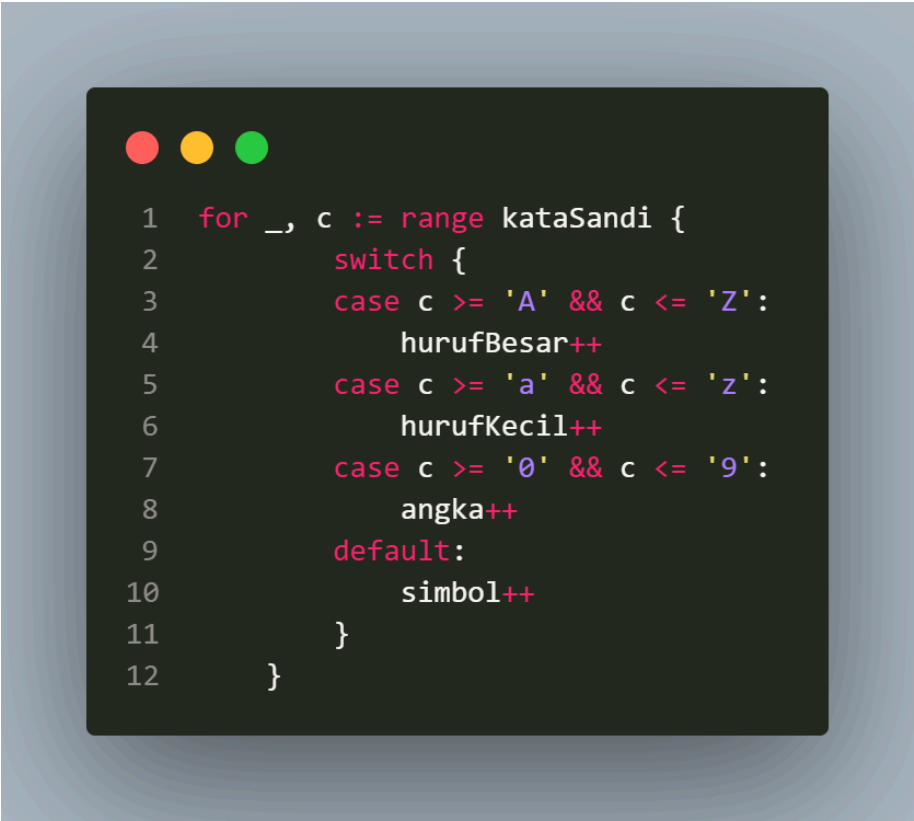
- **struct Account:** Digunakan untuk merepresentasikan sebuah objek akun yang mengelompokkan beberapa data terkait, yaitu ServiceName, Email, Password, dan LastUpdate.
- **var accounts []Account:** Merupakan variabel global bertipe *slice* dari Account. *Slice* digunakan menggantikan *array* statis agar kapasitas penyimpanan data akun dapat bertambah secara dinamis saat fungsi append dipanggil.

2. Perulangan (Looping)

Perulangan digunakan untuk menjaga aplikasi tetap berjalan (menampilkan menu utama) dan untuk menelusuri data akun saat melakukan pencarian, pengurutan, atau penghapusan.



```
1  for {
2      fmt.Println("\n--- SecurePass Menu ---")
3      fmt.Println("1. Tambah Akun")
4      fmt.Println("2. Edit Akun")
5      fmt.Println("3. Hapus Akun")
6      fmt.Println("4. Cari Akun (Sequential/Binary)")
7      fmt.Println("5. Urutkan Akun (Selection/Insertion)")
8      fmt.Println("6. Tampilkan Data & Statistik")
9      fmt.Println("7. Keluar")
10     fmt.Print("Pilihan Anda: ")
11     fmt.Scan(&choice)
```



```
1  for _, c := range kataSandi {
2      switch {
3      case c >= 'A' && c <= 'Z':
4          hurufBesar++
5      case c >= 'a' && c <= 'z':
6          hurufKecil++
7      case c >= '0' && c <= '9':
8          angka++
9      default:
10         simbol++
11     }
12 }
```

Penjelasan:

- **Perulangan for tanpa kondisi (Menu Utama):** Berfungsi sebagai *infinite loop* untuk menjaga program tetap mengeksekusi menu secara berulang hingga pengguna memilih opsi untuk keluar (fungsi return).
- **Perulangan for range:** Digunakan untuk mengiterasi setiap elemen di dalam *slice* accounts secara efisien, baik untuk memeriksa kecocokan data maupun untuk memproses karakter pada sebuah *string* kata sandi.

3. Logika Percabangan (Conditional)

Percabangan digunakan untuk mengontrol alur eksekusi program berdasarkan pilihan menu pengguna, validasi kondisi algoritma, serta penentuan klasifikasi kekuatan kata sandi.


```

1  switch choice {
2      case 1:
3          addAccount()
4      case 2:
5          editAccount()
6      case 3:
7          deleteAccount()
8      case 4:
9          searchMenu()
10     case 5:
11         sortMenu()
12     case 6:
13         displayAccounts()
14     case 7:
15         return
16     default:
17         fmt.Println("Pilihan tidak valid.")
18 }

```

```

1  switch {
2      case skor <= 2:
3          return "Lemah"
4      case skor <= 4:
5          return "Sedang"
6      default:
7          return "Kuat"
8  }

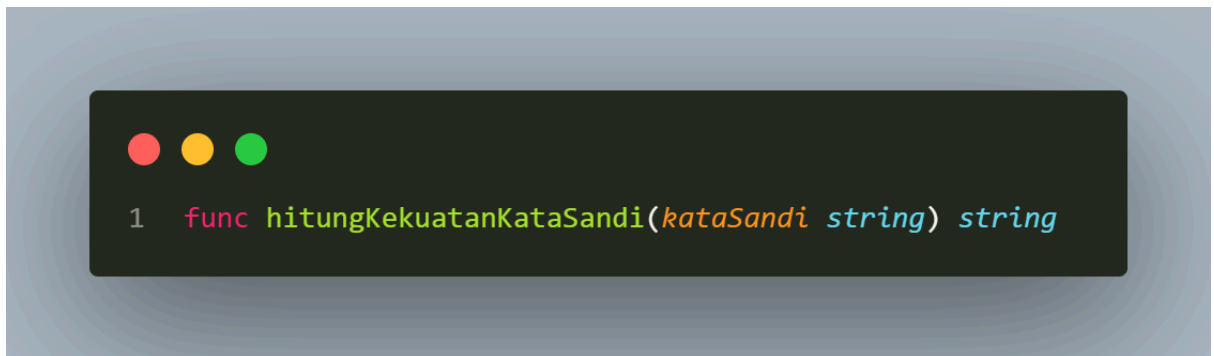
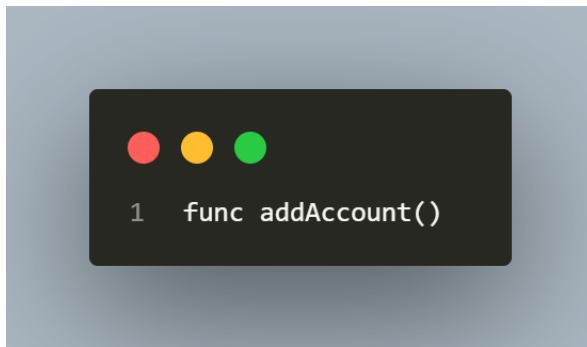
```

Penjelasan:

- **switch-case (Menu Utama):** Menentukan fungsi atau prosedur mana yang akan dieksekusi berdasarkan nilai variabel `choice` yang diinput oleh pengguna.
- **if-else (Pencarian & Validasi):** Digunakan untuk memverifikasi apakah data sudah terurut sebelum menjalankan *Binary Search*, serta menentukan batas atas (high) dan batas bawah (low) pada proses pembagian data.
- **switch (Kekuatan Sandi):** Menentukan kategori akhir ("Lemah", "Sedang", "Kuat") berdasarkan total akumulasi skor karakter yang dipenuhi.

4. Fungsi dan Prosedur Modular

Program ini dipecah menjadi beberapa fungsi dan prosedur fungsional untuk memenuhi prinsip pemrograman modular agar kode lebih terstruktur dan mudah dirawat.



Penjelasan:

- **Prosedur (Fungsi tanpa *return value*):** Seperti `addAccount()`, `sortMenu()`, atau `displayAccounts()`, digunakan untuk menjalankan serangkaian instruksi atau menampilkan data langsung ke layar tanpa mengembalikan nilai ke pemanggilnya.
- **Fungsi dengan *Return Value*:** Seperti `hitungKekuatanKataSandi(kataSandi string) string`, menerima parameter sebuah *string* teks kata sandi, memprosesnya, dan mengembalikan hasil akhir berupa *string* klasifikasi kekuatannya.

BAB IV

TANTANGAN DAN SOLUSI

3.4 Kendala yang Dihadapi dan Solusi Penanganannya

1. Perancangan Validasi pada Algoritma Pencarian (*Binary Search*)

- **Kendala**

Algoritma *Binary Search* mewajibkan seluruh data dalam kondisi terurut secara alfabetis berdasarkan nama layanan agar pencarian titik tengah (*mid-point*) berfungsi dengan benar. Kendala muncul ketika sistem harus memastikan apakah pengguna sudah melakukan pengurutan data terlebih dahulu melalui fitur *Sort Menu* sebelum fitur *Binary Search* dieksekusi. Jika data belum terurut, program akan menghasilkan *output* yang tidak valid atau salah prediksi posisi data.

- **Solusi**

Solusi yang diterapkan adalah dengan menambahkan blok pengecekan kondisi (*validation loop*) di awal pilihan menu *Binary Search*. Sistem akan melakukan iterasi singkat untuk membandingkan setiap elemen `accounts[i].ServiceName` dengan elemen selanjutnya `accounts[i+1].ServiceName`. Jika ditemukan posisi yang tidak berurutan, variabel `Sorted` akan bernilai `false`, lalu program secara otomatis membatalkan proses pencarian dan menampilkan pesan *error* yang meminta pengguna untuk mengurutkan data terlebih dahulu.

2. Runtime Error Saat Implementasi Penghapusan Data (*Slicing*)

- **Kendala**

Pada saat mengimplementasikan fitur Hapus Akun, terjadi kendala berupa *runtime error* atau *out of bounds* saat memotong urutan data di dalam memori. Masalah ini dipicu oleh penggunaan teknik penyuntingan *slice* dinamis: `append(accounts[:i], accounts[i+1:]....)`. Karena indeks *array/slice* bergeser secara otomatis setelah sebuah elemen dihapus, perulangan yang terus berjalan tanpa interupsi sempat membaca indeks yang sudah tidak ada.

- **Solusi**

Solusi untuk mengatasi kendala ini adalah dengan langsung menyisipkan perintah `return` atau `break` tepat setelah baris fungsi `append` berhasil dieksekusi. Dengan menghentikan perulangan secara paksa begitu data ditemukan dan dihapus, program tidak akan melanjutkan iterasi ke indeks berikutnya, sehingga *runtime error* dapat dihindari sepenuhnya.

3. Masalah Validasi Input pada Tipe Data *String* yang Mengandung Spasi

- **Kendala**

Penggunaan fungsi bawaan `fmt.Scan()` menjadi kendala tersendiri ketika pengguna menginputkan nama layanan atau kata sandi yang menggunakan karakter spasi (misalnya: nama layanan "Google Drive"). Fungsi `Scan()` membaca karakter spasi sebagai batas akhir sebuah inputan (*separator*), sehingga kata kedua ("Drive") justru otomatis masuk ke variabel input berikutnya (seperti *Email* atau *Password*) dan merusak alur pengisian data.

- **Solusi**

Solusi sementara yang diterapkan adalah dengan memberikan instruksi yang jelas pada antarmuka pengguna agar memasukkan nama layanan tanpa spasi, atau mengganti spasi menggunakan karakter penghubung seperti *underscore* (contoh: "Google_Drive"). *(Catatan alternatif untuk pengembangan lanjut: fungsi input dapat diganti menggunakan `fmt.Scanln()` atau memanfaatkan library `bufio.NewScanner(os.Stdin)` agar dapat membaca satu baris kalimat utuh termasuk spasi).*

4. Kompleksitas Logika Klasifikasi Kekuatan Kata Sandi

- **Kendala**

Merancang fungsi `hitungKekuatanKataSandi` membutuhkan ketelitian ekstra karena harus mengevaluasi isi *string* karakter demi karakter secara spesifik. Kendala awal yang dihadapi adalah terjadinya salah kalkulasi skor ketika memeriksa kombinasi huruf besar, huruf kecil, angka, dan simbol, sehingga hasil akhir klasifikasi ("Lemah", "Sedang", "Kuat") tidak akurat dan membuat visualisasi data statistik di menu utama menjadi keliru.

- **Solusi**

Solusi yang dilakukan adalah memanfaatkan fitur perulangan `for range` berbasis *rune* untuk membedah setiap karakter pada *string* kata sandi, kemudian mengombinasikannya dengan logika percabangan `switch-case` yang memeriksa rentang nilai ASCII karakter tersebut (misalnya `'A' <= c && c <= 'Z'` untuk huruf besar). Selain itu, aturan penambahan skor dipisahkan ke dalam beberapa kondisi `if` mandiri agar setiap parameter keamanan (panjang karakter dan variasi simbol) mendapatkan bobot nilai yang adil dan tepat.

BAB V

KESIMPULAN DAN REKOMENDASI

5.1 Kesimpulan

Berdasarkan seluruh proses analisis, perancangan, hingga implementasi yang telah dilakukan, tugas besar pengembangan Aplikasi Pengelola Kata Sandi Pribadi (SecurePass) berbasis bahasa Go ini telah berhasil diselesaikan dengan baik. Hasil akhir dari proyek ini menunjukkan bahwa seluruh tujuan awal pengembangan telah tercapai sepenuhnya, di mana mahasiswa mampu mengintegrasikan konsep pemrograman modular, struktur data *struct*, serta algoritma pencarian (*Sequential & Binary Search*) dan pengurutan (*Selection & Insertion Sort*) ke dalam studi kasus nyata.

Secara umum, performa aplikasi SecurePass berjalan dengan sangat stabil, responsif, dan interaktif saat dijalankan melalui antarmuka *Command Line Interface* (CLI). Fitur-fitur utama seperti operasi CRUD akun, pengurutan alfabetis, dan pencarian data dapat dieksekusi tanpa adanya *error* logika yang berarti. Selain itu, fungsi klasifikasi dan kalkulasi statistik kekuatan kata sandi mampu memberikan penilaian yang akurat secara *real-time*, sehingga aplikasi ini dinilai sangat fungsional dalam membantu individu mengelola keamanan data login mereka secara lokal.

5.2 Rekomendasi

Meskipun aplikasi saat ini sudah memenuhi seluruh spesifikasi tugas besar yang dipersyaratkan, terdapat beberapa saran dan rekomendasi yang dapat diterapkan untuk pengembangan aplikasi SecurePass lebih lanjut di masa mendatang:

- **Mengembangkan Aplikasi ke Versi Berbasis GUI atau Web**
Mengubah antarmuka aplikasi dari berbasis teks (CLI) menjadi berbasis grafis (GUI) menggunakan *framework* seperti Fyne (untuk desktop Go) atau mentransformasikannya menjadi aplikasi berbasis web agar tampilan jauh lebih modern, interaktif, dan ramah pengguna (*user-friendly*).
- **Menambahkan Fitur Enkripsi Data Kriptografi**
Meningkatkan aplikasi ini berfungsi untuk mengelola kata sandi, sangat direkomendasikan untuk menambahkan modul enkripsi (misalnya menggunakan algoritma AES atau SHA-256) saat menyimpan data di memori atau lokal, sehingga kata sandi tidak tersimpan dalam bentuk teks biasa (*plain text*) demi keamanan tingkat tinggi.
- **Memperbaiki Mekanisme Validasi Input (Penanganan Spasi)**
Mengoptimalkan efisiensi penangkapan data input dari pengguna dengan mengganti fungsi `fmt.Scan()` menggunakan `bufio.NewScanner(os.Stdin)`. Hal ini penting agar sistem dapat menerima input kalimat yang mengandung spasi secara penuh tanpa merusak urutan variabel data berikutnya.
- **Integrasi Penyimpanan Data Permanen (*File Handling*)**

Menambahkan fitur membaca dan menulis data ke dalam *file* eksternal (seperti format .txt, .json, atau .csv). Dengan demikian, data akun yang telah dimasukkan oleh pengguna tidak akan hilang saat aplikasi ditutup (*persistent storage*).

- **Fitur Generator Kata Sandi Otomatis (*Password Generator*)**

Menambahkan fitur baru yang dapat menghasilkan rekomendasi kata sandi acak yang kuat (kombinasi huruf, angka, dan simbol) secara otomatis ketika pengguna sedang menambahkan akun layanan baru.

TAUTAN GITHUB

Link Repository:

<https://github.com/KelvinAG/TugasBesar-Algoritma-dan-Pemrograman-2>

Di dalam repositori tersebut, struktur penyimpanan berkas telah disusun secara rapi dan sistematis yang meliputi beberapa komponen berikut:

1. **Folder Source Code (/ atau nama folder kode):** Berisi berkas utama kode program berbasis bahasa Go, termasuk berkas `SecurePass.go` yang memuat seluruh implementasi logika fungsi CRUD, algoritma pencarian (*Sequential & Binary*), pengurutan (*Selection & Insertion*), serta fungsi penghitungan klasifikasi kata sandi.
2. **Dokumentasi Laporan:** Berisi berkas cetak atau salinan dokumen laporan perancangan tugas besar (format `.pdf` atau `.docx`) yang menjelaskan gambaran umum aplikasi, pseudocode, analisis kendala, hingga instruksi penggunaan program.
3. **Berkas Pendukung (*Supporting Files*):** Memuat dokumen spesifikasi tugas besar, panduan ringkas cara menjalankan program (*README.md*), serta berkas tambahan lain yang digunakan untuk mempermudah proses kompilasi dan eksekusi program di lingkungan lokal.

REFERENSI

1. Modul Perkuliahan

- Tim Dosen Algoritma dan Pemrograman. (2025). *Modul Kuliah Algoritma dan Pemrograman 2*. Fakultas Informatika, School of Computing, Telkom University.
- Tim Dosen Algoritma dan Pemrograman. (2025). *Panduan Tugas Besar Algoritma dan Pemrograman 2: Aplikasi Pengelola Kata Sandi Pribadi (SecurePass)*. Fakultas Informatika, School of Computing, Telkom University.

2. Dokumentasi Resmi Bahasa Pemrograman

- The Go Authors. (2026). *The Go Programming Language Documentation*. Diakses dari <https://go.dev/doc/> (Referensi resmi untuk penggunaan tipe data *struct*, *slice*, fungsi *append*, dan struktur kontrol perulangan/percabangan pada Golang).
- Go Packages Documentation. (2026). *Package fmt*. Diakses dari <https://pkg.go.dev/fmt> (Referensi resmi untuk implementasi fungsi input/output seperti *fmt.Scan* dan *fmt.Printf*).